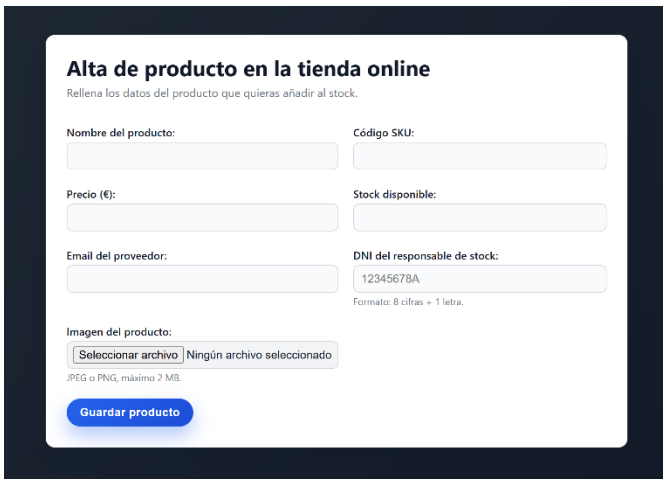
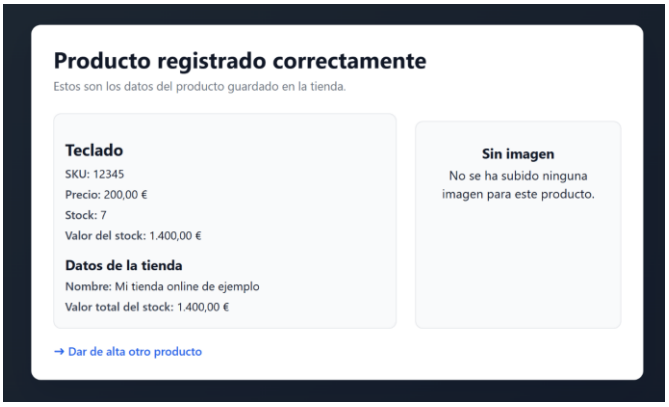
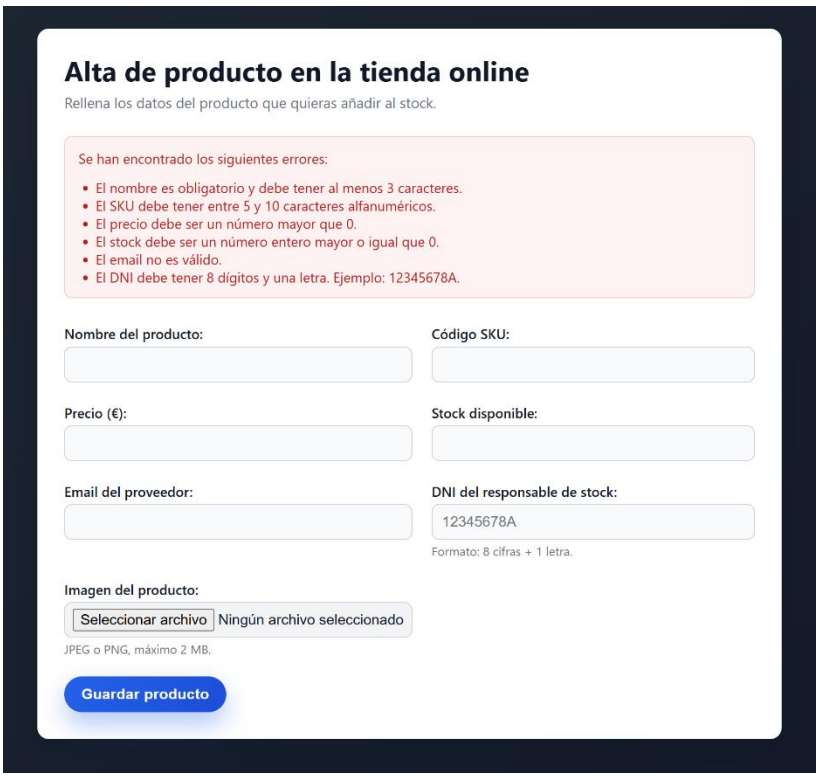
Examen de PHP – Formularios, validación, POO (básico)







1. Descripción general

Implementar una mini aplicación web para **dar de alta un producto** en el stock de una tienda online.

La aplicación deberá:

1. Mostrar un **formulario** para introducir los datos de un producto.
2. **Validar** los datos recibidos del formulario en el servidor (PHP).
3. Si hay errores, mostrar el formulario con los mensajes de error.
4. Si los datos son correctos:
    - Subir una **imagen de producto**, comprobando su **tipo MIME**.
    - Crear objetos de clases PHP (POO sencilla, sin herencia).
    - Mostrar un **resumen del producto** y algunos datos de la tienda (usando métodos de las clases).

Además, el ejercicio debe seguir una **estructura tipo MVC simple**:

- `index.php` → Punto de entrada.
- Carpeta `classes/` → Clases PHP.
- Archivo `functions.php` → Funciones de validación y ayuda.
- Carpeta `views/` → Vistas para el formulario y para el resumen.

---

2. Estructura de ficheros obligatoria

Usando, como mínimo, estos archivos y carpetas:

```
index.php
functions.php
/classes
    Producto.php
    Tienda.php
/views
    formulario.php
    resumen.php
/uploads    (carpeta para guardar imágenes subidas)
```

---

3. Campos del formulario (ya está implementado pero leer igualmente)

En `views/formulario.php` hay un formulario (html) que envíe por `POST` a `index.php` y permita introducir:

1. **Nombre del producto** (`nombre`) – *obligatorio*
   - Mínimo 3 caracteres.
2. **Código SKU** (`sku`) – *obligatorio*
   - Solo caracteres alfanuméricos. *(pista: preg_match A-Z0-9)*
   - Entre 5 y 10 caracteres.
3. **Precio** (`precio`) – *obligatorio*
   - Número mayor que 0.
4. **Stock disponible** (`stock`) – *obligatorio*
   - Número entero mayor o igual que 0. (se puede usar `filter_var` con `FILTER_VALIDATE_INT`).
5. **Email del proveedor** (`email`) – *obligatorio*
   - Debe ser un email válido (usa `filter_var` con `FILTER_VALIDATE_EMAIL`).
6. **DNI del responsable de stock** (`dni`) – *obligatorio*
   - Formato: 8 dígitos y 1 letra final.
   - Ejemplo válido: `12345678A`.
   - Para validarlo debes usar la función `preg_match` con este patrón:
   - `$patronDni = '/^[0-9]{8}[A-Za-z]$/';`
   - `// Ejemplo de uso:`
   - `// preg_match($patronDni, $dni)`
7. **Imagen de producto** (`imagen`) – *campo de tipo file*
   - Aceptar **solo** imágenes **JPEG o PNG**.
   - Debes comprobar el **tipo MIME** usando `$_FILES['imagen']['type']`.

- Tamaño máximo: 2 MB (2 * 1024 * 1024 bytes).
- Si no se sube ninguna imagen, no debe considerarse error, es opcional.

El formulario debe utilizar `enctype="multipart/form-data"`.

---

4. Validación y funciones (`functions.php`)

En el archivo `functions.php` debes crear al menos:

1. Una función para **limpiar** los datos de texto recibidos:

```
function limpiarDato(string $dato): string {
    // Solo debe quitar espacios y escapar caracteres especiales
(HTML)
   }
```

2. Una función principal para validar el formulario, por ejemplo:

```
function validarFormulario(array $datos, ?array $imagen): array
{
       // Devuelve un array de errores:
       //  - clave = nombre del campo
       //  - valor = mensaje de error
       // Si no hay errores, devuelve un array vacío.
   }
```

   Dentro de esta función deberás:

   - Comprobar cada campo con las reglas indicadas.
   - Para el DNI, usar `preg_match` con el patrón indicado en el enunciado.
   - Para la imagen:
     - Si hay fichero subido `UPLOAD_ERR_NO_FILE`, comprobar:
       - `error` (utilizando `UPLOAD_ERR_OK`).
       - `type` sea `image/jpeg` o `image/png`.
       - `size` no supere 2 MB.
3. Una función para **guardar la imagen** en la carpeta `uploads` (si se ha subido y es válida), por ejemplo:

```
  function guardarImagen(array $imagen, string $directorio):
  ?string {
  // En la carpeta "uploads"…
  // Guarda el archivo con un nombre único (con la fecha + su
nombre de archivo, por ejemplo.)
      // y devuelve la ruta relativa o null si falla.
  }
```

---

5. Clases POO (carpeta `classes/`)

5.1 Clase `Producto` (archivo `classes/Producto.php`)

Implementa una clase `Producto` con:

- **Atributos privados** mínimos:
  - `nombre` (string)
  - `sku` (string)
  - `precio` (float)
  - `stock` (int)
  - `rutaImagen` (string o null) → ruta del fichero subido
- **Constructor** con parámetros para inicializar todos los atributos.
- **Getters** para acceder a los atributos (al menos para todos los anteriores).
- Un método:
- `public function getValorStock(): float`

  que devuelva `precio * stock`.

- Dos métodos sencillos para modificar el stock:
- `public function añadirStock(int $cantidad): void`
- `public function reducirStock(int $cantidad): void`
  - `añadirStock` suma la cantidad al stock si es > 0.
  - `reducirStock` resta la cantidad si es > 0 y no deja el stock en negativo.

5.2 Clase `Tienda` (archivo `classes/Tienda.php`)

Implementa una clase `Tienda` con:

- **Atributos privados**:
  - `nombre` (string)
  - `productos` (array de objetos `Producto`)
- **Constructor** que reciba el nombre de la tienda.
- Método para añadir productos:
- `public function agregarProducto(Producto $producto): void`
- Método para obtener todos los productos:
- `public function getProductos(): array`
- Método para calcular el **valor total** del stock:
- `public function getValorTotalStock(): float`
  - Suma el `getValorStock()` de todos los productos.

No se permite herencia en este ejercicio. Solo estas dos clases y sus métodos.

---

6. Lógica en `index.php`

El archivo `index.php` actuará como "controlador" simple:

1. Incluirá los archivos necesarios:

    - `functions.php`
    - `classes/Producto.php`
    - `classes/Tienda.php`
2. Si la petición es `GET`:
    - Preparará un array de datos vacíos y un array de errores vacío.
    - Incluirá la vista `views/formulario.php` para mostrar el formulario.
3. Si la petición es `POST`:
    - Recogerá los datos del formulario.
    - Limpiará los datos con `limpiarDato`.
    - Llamará a `validarFormulario(...)`.
    - Si hay errores:
        - Mostrará de nuevo el formulario (`views/formulario.php`) con los errores y los valores introducidos.
    - Si no hay errores:
        - Guardará la imagen con `guardarImagen(...)`.
        - Creará un objeto `Producto` con los datos correctos.
        - Creará un objeto `Tienda` (por ejemplo con nombre fijo: `"Mi tienda online"`).
        - Añadirá el producto a la tienda (`agregarProducto`).
        - Cargará la vista `views/resumen.php`, donde se mostrará:
            - Datos del producto (usando getters de `Producto`).
            - Valor del stock del producto.
            - Nombre de la tienda y valor total del stock (usando métodos de `Tienda`).
            - La imagen del producto si se ha subido.

---

7. Vistas (`views/formulario.php` y `views/resumen.php`)

- `views/formulario.php` debe:
    - Mostrar el formulario con los campos indicados.
    - Si hay errores (array `$errores`), mostrarlos (por ejemplo en una lista).
    - Rellenar de nuevo los campos con los valores enviados si había errores.
- `views/resumen.php` debe:
    - Mostrar un mensaje tipo “Producto registrado correctamente”.
    - Mostrar los datos del producto y de la tienda usando métodos de los objetos.
    - Mostrar la imagen si existe ruta de imagen.
    - Ofrecer un enlace para volver a `index.php` y crear otro producto.

Bloque	Qué evalúo principalmente	Rango de puntos
1	**Estructura general y flujo (index.php + incluir)**	2,5
2	**Validación y tratamiento de datos (functions.php, sin imagen)**	3,0
3	**Gestión de imagen de producto (validación + guardado)**	1,5
4	**POO: clases Producto y Tienda (atributos, constructor, métodos básicos)**	2,5
5	**Vistas (formulario + resumen) y uso de datos**	0,5
6	**Calidad global y funcionamiento (que corra sin romperse, orden, mínimos detalles)**	0,5